张利顺-工作交接文档

网络库@王强 (王强-开发) @咸光金 @张文彬

设计文档： Android 网络库设计文档

使用文档： Android网络库使用说明

Retrofit多实例

- ipv4api一个实例
- privacyTrackApi一个实例
- ossApiService2一个实例

配置https证书

Okhttp配置证书ZLSSLSocketFactor，debug包信任所有证书，relase包读取本地和远端证书

证书更新检查com.zhaopin.social.base.c.autoupdate.UpdateUtil#requestUpdate

Invocation

Retrofit在创建request的之后给okhttp添加的一个tag，Retrofit服务接口方法的单个调用。该类捕获被调用的方法和该方法的参数。在OKHttp拦截中只有使用接口声明的时候才有

```
    List<Object> argumentList = new ArrayList<>(argumentCount);
    for (int p = 0; p < argumentCount; p++) {
      argumentList.add(args[p]);
      handlers[p].apply(requestBuilder, args[p]);
    }

    return requestBuilder.get().tag(Invocation.class, new Invocation(method, argumentList)).build();
```

ToStringConverterFactory

用于配置到Retrofit的addConverterFactory上；

ResponseBody转换成String=>返回值支持Call<String> ，suspend String， Observable<String>

FastJsonConverterFactory

用户配置Retrofit的addConverterFactory上；

ResponseBody转换成Bean=>返回值支持Call<Bean> ，suspend Bean， Observable<Bean>

Bean转换成RequestBody=>需要指定@Body注解

```
@POST("load/put")
suspend fun request(@Body user: User) : Bean

@POST("load/put")
suspend fun request2(@Body map: MutableMap<String,String>) : Bean
```

ApiResponseCallAdapterFactory

适配接口返回返回值类型，即returnType。

```
class ApiResponseCallAdapterFactory private constructor() : CallAdapter.Factory() {

    override fun get(
        returnType: Type,
        annotations: Array<out Annotation>,
        retrofit: Retrofit
    ): CallAdapter<*, *>? {
        return null
    }

    companion object {
        fun create(): ApiResponseCallAdapterFactory = ApiResponseCallAdapterFactory()
    }
}

sealed class ApiResponse
```

协程取消的问题

```
private val mainScope = MainScope()

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(binding.root)

```

张利顺 2860 北京网聘信息技术有限公司

```
    mainScope.launch {
        withContext(NonCancellable) {
            delay(3000)
        }
        try {
            val userSuspend2 = githubApi.getUserSuspend2("ihrthk")
            println("userSuspend2 = $userSuspend2")
        } catch (e: Exception) {
            e.printStackTrace()
            //解决方案
            if (e is CancellationException) {
                throw e
            }
        }
        //问题 协程取消后，这个代码块仍然会执行
        println("after cancel do something")
    }

    binding.btnEmit.setOnClickListener {
        mainScope.cancel()
    }
}
```

整个请求过程

1. retrofit.create(PermissionApi::class.java)

检查接口类声明(是否是一个接口，泛型判断)

使用动态代理创建PermissionApi的实现类

2.ServiceMethod.parseAnnotations(this, method)

使用反射解析指定的接口方法，放到map缓存里，再次调用相同接口可以直接使用。

```
//Retrofit.java
private final Map<Method, ServiceMethod<?>> serviceMethodCache = new ConcurrentHashMap<>();
```

3.选择合适的CallAdatper

配置代码

张利顺 2860 北京网聘信息技术有限公司

```
//Network.java
.addCallAdapterFactory(new LiveDataCallAdapterFactory())
.addCallAdapterFactory(RxJava2CallAdapterFactory.createWithScheduler(Schedulers.io()))
//Retrfoit
CompletableFutureCallAdapterFactory
DefaultCallAdapterFactory
```

获取逻辑：从容器的下标0开始，依次匹配

```
//HttpServiceMethod.java
CallAdapter<ResponseT, ReturnT> callAdapter =
        createCallAdapter(retrofit, method, adapterType, annotations);
//1.retrofit参数，用于遍历callAdapterFactories
//2.method参数，仅用于异常信息的提示
//3.adapterType，返回值类型
//4.annotations，方法上的注解

//Retrofit.java
public CallAdapter<?, ?> nextCallAdapter(
  @Nullable CallAdapter.Factory skipPast, Type returnType, Annotation[] annotations) {
  Objects.requireNonNull(returnType, "returnType == null");
  Objects.requireNonNull(annotations, "annotations == null");

  int start = callAdapterFactories.indexOf(skipPast) + 1;
  for (int i = start, count = callAdapterFactories.size(); i < count; i++) {
    CallAdapter<?, ?> adapter = callAdapterFactories.get(i).get(returnType, annotations, this);
    if (adapter != null) {
      return adapter;
    }
  }
}
```

4.选择合适的Converter.Factory

配置代码

```
//Retrofit.java
converterFactories.add(new BuiltInConverters());
```

```
//Network.java
.addConverterFactory(new ToStringConverterFactory())
.addConverterFactory(FastJsonConverterFactory.create())
//Retrofit.java
OptionalConverterFactory
```

获取逻辑

```
//HttpServiceMethod.java
Converter<ResponseBody, ResponseT> responseConverter =
        createResponseConverter(retrofit, method, responseType);
//1.retrofit参数，用于遍历callAdapterFactories
//2.method参数，仅用于异常信息的提示
//3.responseType参数，返回值类型

//Retrofit.java
public <T> Converter<ResponseBody, T> nextResponseBodyConverter(
      @Nullable Converter.Factory skipPast, Type type, Annotation[] annotations) {
    Objects.requireNonNull(type, "type == null");
    Objects.requireNonNull(annotations, "annotations == null");

    int start = converterFactories.indexOf(skipPast) + 1;
    for (int i = start, count = converterFactories.size(); i < count; i++) {
      Converter<ResponseBody, ?> converter =
          converterFactories.get(i).responseBodyConverter(type, annotations, this);
      if (converter != null) {
        //noinspection unchecked
        return (Converter<ResponseBody, T>) converter;
      }
    }
  }
```

5.选择合适HttpServiceMethod的子类

支持Java的Call和 Kotlin的suspend；两种语言返回类型都支持Response和ResponseBody，它们俩的区别是如果声明的是ResponseBody，并且发生 Http 错误，则会抛出HttpException。

```
    //HttpServiceMethod.java
    if (!isKotlinSuspendFunction) {
```

```
      return new CallAdapted<>(requestFactory, callFactory, responseConverter, callAdapter);
    } else if (continuationWantsResponse) {
      //noinspection unchecked Kotlin compiler guarantees ReturnT to be Object.
      return (HttpServiceMethod<ResponseT, ReturnT>)
          new SuspendForResponse<>(
              requestFactory,
              callFactory,
              responseConverter,
              (CallAdapter<ResponseT, Call<ResponseT>>) callAdapter);
    } else {
      //noinspection unchecked Kotlin compiler guarantees ReturnT to be Object.
      return (HttpServiceMethod<ResponseT, ReturnT>)
          new SuspendForBody<>(
              requestFactory,
              callFactory,
              responseConverter,
              (CallAdapter<ResponseT, Call<ResponseT>>) callAdapter,
              continuationBodyNullable);
    }
```

Java使用DefaultCallAdapterFactory的切换到到主线程，Kotlin使用的是恢复拦截切到主线程

```
public void enqueue(final Callback<T> callback) {
  Objects.requireNonNull(callback, "callback == null");

  delegate.enqueue(
      new Callback<T>() {
        @Override
        public void onResponse(Call<T> call, final Response<T> response) {
          callbackExecutor.execute(
              () -> {
                if (delegate.isCanceled()) {
                  // Emulate OkHttp's behavior of throwing/delivering an IOException on
                  // cancellation.
                  callback.onFailure(ExecutorCallbackCall.this, new IOException("Canceled"));
                } else {
                  callback.onResponse(ExecutorCallbackCall.this, response);
```

张利顺 2860 北京网聘信息技术有限公司

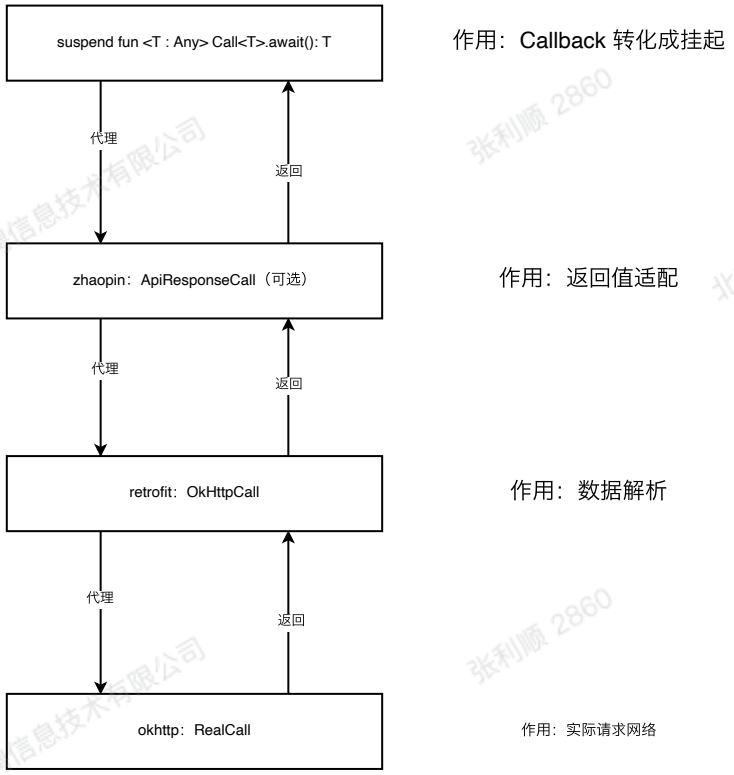
```
                }
            });
        }

        @Override
        public void onFailure(Call<T> call, final Throwable t) {
          callbackExecutor.execute(() -> callback.onFailure(ExecutorCallbackCall.this, t));
        }
      });
}
```

6.结合解析注解的结果和用户传递的实参创建接口的Request

```
  //构造方法
    OkHttpCall(
      RequestFactory requestFactory,
      Object[] args,
      okhttp3.Call.Factory callFactory,
      Converter<ResponseBody, T> responseConverter) {
    this.requestFactory = requestFactory;
    this.args = args;
    this.callFactory = callFactory;
    this.responseConverter = responseConverter;
  }
  //创建Request
  private okhttp3.Call createRawCall() throws IOException {
    okhttp3.Call call = callFactory.newCall(requestFactory.create(args));
    if (call == null) {
      throw new NullPointerException("Call.Factory returned null.");
    }
    return call;
  }
```

7.发起网络请求整体流程(HttpCall的代理模式)

张利顺 2860 北京网聘信息技术有限公司

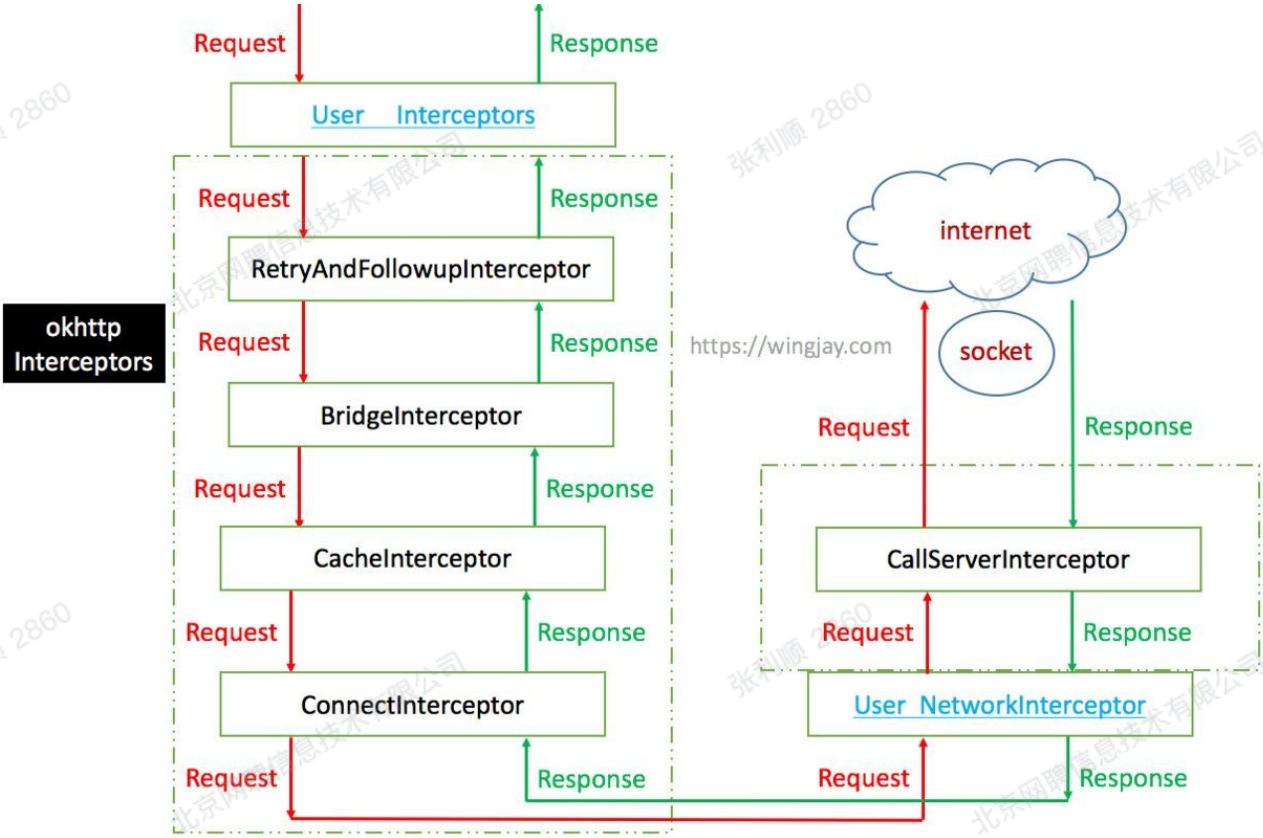



```java
//Retrofit.create()
public @Nullable Object invoke(Object proxy, Method method, @Nullable Object[] args)
    throws Throwable {
  // If the method is a method from Object then defer to normal invocation.
  if (method.getDeclaringClass() == Object.class) {
    return method.invoke(this, args);
  }
  args = args != null ? args : emptyArgs;
  return platform.isDefaultMethod(method)
  ? platform.invokeDefaultMethod(method, service, proxy, args)
  : loadServiceMethod(method).invoke(args);
}
//HttpServiceMethod.java
final @Nullable ReturnT invoke(Object[] args) {
  Call<ResponseT> call = new OkHttpCall<>(requestFactory, args, callFactory, responseConverter);
  return adapt(call, args);
}
//SuspendForBody.java
protected Object adapt(Call<ResponseT> call, Object[] args) {
```

```
  call = callAdapter.adapt(call);

  //noinspection unchecked Checked by reflection inside RequestFactory.
  Continuation<ResponseT> continuation = (Continuation<ResponseT>) args[args.length - 1];

  // Calls to OkHttp Call.enqueue() like those inside await and awaitNullable can sometimes
  // invoke the supplied callback with an exception before the invoking stack frame can return.
  // Coroutines will intercept the subsequent invocation of the Continuation and throw the
  // exception synchronously. A Java Proxy cannot throw checked exceptions without them being
  // declared on the interface method. To avoid the synchronous checked exception being wrapped
  // in an UndeclaredThrowableException, it is intercepted and supplied to a helper which will
  // force suspension to occur so that it can be instead delivered to the continuation to
  // bypass this restriction.
  try {
    return isNullable
    ? KotlinExtensions.awaitNullable(call, continuation)
    : KotlinExtensions.await(call, continuation);
  } catch (Exception e) {
    return KotlinExtensions.suspendAndThrow(e, continuation);
  }
}
```

8.RealCall请求网络，执行OkHttp的五大拦截器及公共拦截器

张利顺 2860 北京网聘信息技术有限公司



passport模块 @张文彬 @曹海生

登录模块技术设计

登录模块

登录模块是最早使用Kotlin代码，PWVerifyActivity是主登录页面，可以进行 BC的帐密和验证码登录。AliOneLoginTools的loginAuth是一键登录的入口。

气泡组件 @咸光金

气泡组件

项目地址： https://gitlab.dev.zhaopin.com/Mobile/CommonWidget

BubbleActivity是Demo测试页面，主要提供BubbleView气泡引导自定义View，还有BubblePopupWindow气泡浮窗类。

工具类优化 @咸光金

Android 工具类优化一期影响范围

Android 工具类优化二期影响范围

项目地址：https://gitlab.dev.zhaopin.com/Mobile/android_module_common_util

优化的工具类有：FileUtils、Storage、DateUtil 、TimeUtil、NetworkUtil、StringUtils、SystemUtil、RegexUtils、NoticationUtils、DownloadUtils、ScreenUtils、StatusBarUtils、KeyboardUtils、BitmapUtil和ImageUtils

oss接口替换@张文彬

阿里云OSSSDK使用接口替换的影响范围

IOssProvider是对外提供的接口类，OssProvider为实现类。支持上传文件、下载文件能力。使用单独的Retrofit和Okhttp实例实现，通过拦截器实现上传进度和下载进度的回调。

share模块@张文彬

对外提供的Service： ✅分享

IShareProvider是对外提供的接口类，ShareService为实现类。支持微信、钉钉、微博、QQ 分享平台分享，支持的格式有图片、文本和网页。除了分享能力，还有包括一个底部分享的弹窗。

基础数据@王强 (王强-开发)

项目地址：https://gitlab.dev.zhaopin.com/Mobile/zpm-select-widget

模块名称：basedata

设计和使用文档： 基础数据设计和使用文档

BaseDataManager是对外提供的类，里面的实现是用Kotlin及协程做的。Demo里面有一个『基础数据』的选项，可以用来调试。运行Main.kt可以用来进行本地的基础数据更新。

支付模块@张文彬

支付Service文档： 支付Service

相关类名：PayService

IPayProvider是对外提供的接口类，PayService为实现类。对接了微信支付和支付宝支付，注意判断微信和支付宝是否安装要是用这个类提供的方法(避免隐私合规问题)。

校园闪屏 @王强 (王强-开发)

页面类名：SchoolSplashActivity

需求背景：https://wiki.zhaopin.com/pages/viewpage.action?pageId=140494032

需求描述：校园身份，切换从到后台切换到前台，满足下列条件会展示广告

1.停留后台超过指定时间(默认90s)

2.小于每天三次

3.图片下载小于800ms

抖音快手返回 @王强 (王强-开发)

相关类名：BackUrlDeepLinkUtil

需求背景：https://wiki.zhaopin.com/pages/viewpage.action?pageId=69253235

需求描述：用户在头条系产品点击广告，调起对应APP后，到达的页面内有返回按钮，可以点击返回调起前头条系产品的页面

独立运行RunAlone @咸光金

插件名称：com.zhaopin.zpm:run_alone:0.0.1

项目地址：https://gitlab.dev.zhaopin.com/Mobile/android-zpm-runalone

设计使用文档：https://wiki.zhaopin.com/pages/viewpage.action?pageId=53569075

张利顺 2860 北京网聘信息技术有限公司